

RECURSIVE BINARY SEARCH:
ALGORITHM, EXAMPLE & TIME
COMPLEXITY ANALYSIS

Presented By:

MEENAKSHI SARKAR

University Roll No – 21301221088

THE HERITAGE ACADEMY

10TH AUGUST 2023

INTRODUCTION

Binary Search is the search technique that works efficiently on sorted lists. Hence, to search an element into some list using the binary search technique, we must ensure that the list is sorted.

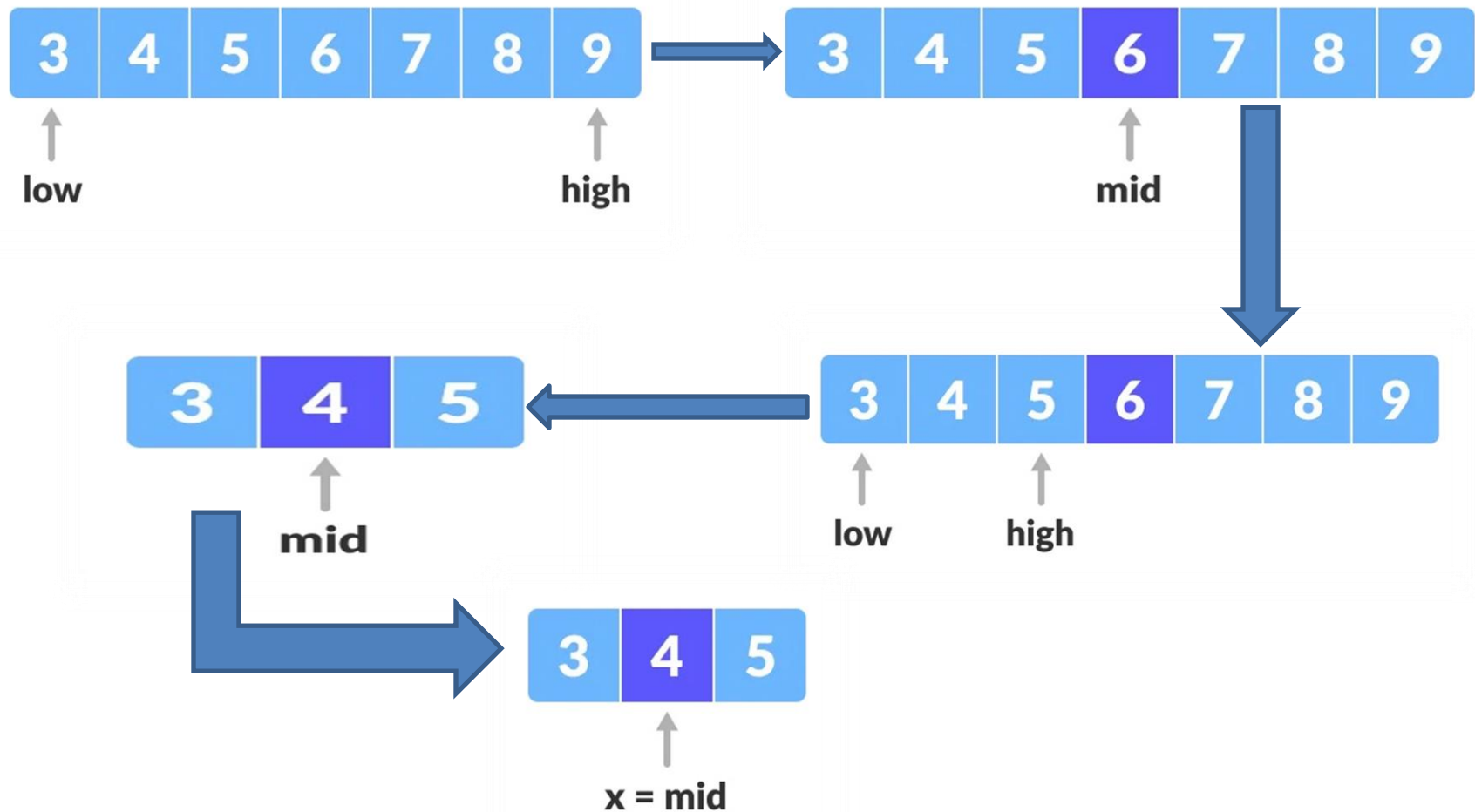
Recursive Binary Search is an implementation of the binary search algorithm that uses recursive method calls (instead of iteratively searching for the item within a single method call).

THE ALGORITHM

Algorithm recursive_binary_search(arr, x, low, high)

```
{  
    if low <= high  
        mid = (low + high) / 2  
        if arr[mid] = x  
            return mid # Target found at index mid  
        else if arr[mid] < x  
            return recursive_binary_search(arr, target, mid + 1, high) # Search in the right half  
        else  
            return recursive_binary_search(arr, target, low, mid - 1) # Search in the left half  
    else  
        return # Target not found  
}
```

EXAMPLE OF RECURSIVE BINARY SEARCH



TIME COMPLEXITY ANALYSIS

Best Case Time Complexity: $O(1)$:

Best case is when the element is at the middle index of the array. It takes only one comparison to find the target element. So the best case complexity is **$O(1)$** .

Average Case Time Complexity: $O(\log N)$:

The average complexity = $(N * (\log N - 1) + 1) / (N + 1) = N * \log N / (N + 1) + 1 / (N + 1)$. Here the dominant term is $N * \log N / (N + 1)$ which is approximately **$\log N$** . So the average case complexity is **$O(\log N)$** .

Worst Case Time Complexity: $O(\log N)$:

The worst case will be when the element is present in the first position. As seen in the average case, the comparison required to reach the first element is **$\log N$** . Worst case may also occur if the value is at the last index of the array. So the time complexity for the worst case is **$O(\log N)$** .

CALCULATE TIME COMPLEXITY:

$$T(n)=1, \text{ when } n=1;$$

$$T(n)=T(n/2)+1, \text{ when } n>1;$$

$$T(n)= T(n/2)+1\text{.....for 1st time}$$

$$T(n/2)=\{T(n/4)+1\}+1$$

$$=T(n/4)+2\text{.....for 2nd time}$$

$$T(n/4)=T(n/8)+3$$

$$=T(n/2^3)+3$$

....

....

Continue upto k times

$$\text{So, } T(n)=T(n/2^k)+k\text{.....(i)}$$

$$\text{Assume, } T(n/2^k)=1$$

$$\Rightarrow n/2^k=1$$

$$\Rightarrow 2^k=n$$

$$\Rightarrow k=\log_2 n$$

Putting the value of k in equation (i)

$$T(n)=1+\log_2 n$$

$$T(n)=\log_2 n$$

ADVANTAGES OF **RECURSIVE BINARY SEARCH**

- **Efficiency**
- **Simple Implementation**
- **Memory Efficiency**
- **Optimal for Sorted Data**
- **Divide and Conquer**

DISADVANTAGES OF RECURSIVE BINARY SEARCH

- **Recursive Overhead**
- **Implementation Complexity**
- **Limited Applicability**
- **Not Always Faster**
- **Lack of Flexibility**

CONCLUSION

In conclusion, we must remember that the future of any algorithm's scope depends on technological advancements, emerging problem domains, and the needs of various industries. While recursive binary search is already a well-established technique, ongoing research and innovation could lead to exciting new applications and optimizations.

And we should always remember few points such as:

- Efficient Precision
- Divide and Conquer
- Logarithmic Complexity
- Recursive Logic
- Space Efficiency
- Versatile Technique
- Real-world Impact
- Mastering Through Practice
- Algorithmic Elegance
- Precision and Speed [1]

REFERENCES

1. <https://www.techiedelight.com/binary-search/>
[Access Date: 08.08.23 Time: 2:30pm]
2. [https://www.chegg.com/homework-help/definitions/recursive-binary-search-3#:~:text=Recursive%20binary%20search%20is%20an,within%20a%20single%20method%20call\).](https://www.chegg.com/homework-help/definitions/recursive-binary-search-3#:~:text=Recursive%20binary%20search%20is%20an,within%20a%20single%20method%20call).)
[Access Date: 08.08.23 Time: 3:30pm]

THANK YOU